Теперь опять сверяем вложенность! (в процессе анализа кода я скорректировал некоторые строки по отступу так как изза некоррктного отступа они были на уровне def).
Возможноесть другие ошибки с отступами.
ОБЯЗАТЕЛЬНО УКАЗЫВАЙ НОМЕРА СТРОК КОТОРЫЕ НУЖНО ИЗМЕНИТЬ! А ТАКЖЕ НОМЕР 1 И НОМЕР ПОСМЛЕДНЕЙ СТРОК КОТОРЫЕ ОСТАЮТЬСЯ КАК ЕСТЬ!

```
353
354     def __call2__(self, program_text: str):
355         autograd_ctx=self._autograd_ctx #захватываем ссылку на autograd
356         # 1. Захватываем область видимости из фрейма вызова forward()
357         caller_locals = sys._getframe(1).f_locals
358         caller_globals = sys._getframe(1).f_globals
359         self._captured_locals = caller_locals
360 
361         # Сбрасываем кэш захваченных тензоров
362         self._captured_inputs.clear()
363         self._captured_outputs.clear()
364 
365         # Разделяем входную строку по символу переноса "\n"
366         raw_lines = program_text.strip().split('\n')
367         compiled_statements = []
368         compiled_backward_statements = []  # Для накопления строк обратного прохода
369         detected_tensors_set = set()
370 
371         for line in raw_lines:
372             cleaned_line = line.strip()
373             if not cleaned_line or cleaned_line.startswith('#'):
374                 continue
375 
376             # Заменяем все "nnclloc." на "nn_"
377             cleaned_line = cleaned_line.replace('nnclloc.', 'nn_')
378 
379             # Вплавляем значения скалярных параметров (например, q = 0.23 -> /=0.23)
380             # Ищем потенциальные имена переменных в строке
381             tokens = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', cleaned_line)
382             for token in tokens:
383                 if token in ['nn_', 'sigmoid', 'tanh', 'dropout', 'tid']:
384                     continue
385                 # Если токен найден в локальных переменных и это число (int, float)
386                 if token in caller_locals and isinstance(caller_locals[token], (int, float)):
387                     val = caller_locals[token]
388                     # Безопасно заменяем имя переменной на её числовое значение
389                     cleaned_line = re.sub(r'\b' + token + r'\b', f"{val}", cleaned_line)
390 
391             # --- БЛОК ВАЛИДАЦИИ ИНСТРУКЦИИ ---
392 
393             # Шаг 1: Проверяем наличие хотя бы одного реального тензора в строке
394             line_tokens = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', cleaned_line)
395             has_tensor = False
396             for t in line_tokens:
397                 if t in caller_locals and isinstance(caller_locals[t], Tensor):
398                     has_tensor = True
399                     detected_tensors_set.add(t)
400 
401             if not has_tensor:
402                 raise ValueError(f"Ошибка валидации: Строка '{line}' не содержит ни одного тензора.")
403 
404             target_tensor_name = None
405 
406             # Шаг 2: Проверяем наличие знака равенства "="
407             if '=' not in cleaned_line:
408                 # Если знака равенства нет, должно быть строго ++ или --
```

```
        # Шаг 2: Проверяем наличие знака равенства "="
        if '=' not in cleaned_line:
            # Если знака равенства нет, должно быть строго ++ или --
            if '++' not in cleaned_line and '--' not in cleaned_line:
                raise SyntaxError(f"Ошибка валидации: Строка '{line}' не содержит знака '=' или инкремента/декремента (++ / --).")

            # Извлекаем имя тензора, к которому применен инкремент/декремент
            match_inc_dec = re.search(r'\b([a-zA-Z_][a-zA-Z0-9_]*)\b\s*(?:\+\+|--)', cleaned_line)
            if match_inc_dec:
                target_tensor_name = match_inc_dec.group(1)
        else:
            # Шаг 3: Разделяем строку по знаку равенства
            left_part, right_part = cleaned_line.split('=', 1)
            left_part = left_part.strip()
            right_part = right_part.strip()

            # Проверяем комбинированные операторы присваивания (+=, -=, *=, /=, %=)
            # Если перед '=' стоит один из знаков арифметики
            if left_part and left_part[-1] in ['+', '-', '*', '/', '%']:
                target_tensor_name = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', left_part)[-1]
            else:
                # Шаг 4: Чистое присваивание '=', проверяем правую часть на валидность данных
                target_tensor_name = left_part

                # Извлекаем все буквенные токены из правой части, кроме вызовов функций nn_
                right_tokens = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', right_part)
                for rt in right_tokens:
                    if rt.startswith('nn_') or rt in ['sigmoid', 'tanh', 'dropout', 'tid']:
                        continue
                    # Если в правой части осталась строка, которая не является тензором и не была вплавлена как число
                    if rt in caller_locals and isinstance(caller_locals[rt], Tensor):
                        continue
                    raise TypeError(f"Ошибка валидации: В правой части выражения '{line}' обнаружено недопустимое строковое значение '{rt}'.")

        if not target_tensor_name:
            raise ValueError(f"Ошибка валидации: Не удалось определить целевой тензор в строке '{line}'.")

        self._captured_outputs.add(target_tensor_name)
        # --- ГЕНЕРАЦИЯ СТРОКИ ДЛЯ BACKWARD ПО ТВОЕМУ ПЛАНУ ---
        if self.is_training:
            # Дублируем очищенную строку (например, Y=nn_tanh(X))
            bwd_line = cleaned_line

            if "nn_dropout" in bwd_line:
                # Исключение для dropout: заменяем функцию на nn_dropout_backward,
                # заменяя первый аргумент X на его градиент grad_X (входной градиент идет вместо X)
                # Формат: nn_dropout_backward(grad_Y, p, tid, seed)
                bwd_line = bwd_line.replace("nn_dropout", "nn_dropout_backward")
                # Заменяем целевой таргет на градиент источника
                compiled_bwd = f"if (tid < {target_tensor_name}Size) {{ grad_{target_tensor_name}[tid] += {bwd_line.split('=', 1)[1]}; }}"
            else:
                # Стандартный паттерн: ищем буквы между "nn_" и "("
                match_func = re.search(r'nn_([a-zA-Z0-9_]+)\(', bwd_line)
                if match_func:
                    func_name = match_func.group(1)  # получили к примеру 'tanh'
```

```
                    match_func = re.search(r'nn_([a-zA-Z0-9_]+)\(', bwd_line)
                    if match_func:
                        func_name = match_func.group(1)  # получили к примеру 'tanh'
                        # Вставляем _backward перед скобкой
                        bwd_line = bwd_line.replace(f"nn_{func_name}(", f"nn_{func_name}_backward(")

                        # Находим аргумент внутри скобок, чтобы узнать имя входного тензора
                        match_args = re.search(r'nn_[a-zA-Z0-9_]+_backward\((.*?)\)', bwd_line)
                        if match_args:
                            inner_arg = match_args.group(1).split(',')[0].strip() # имя тензора X
                            # Перед закрывающей скобкой вставляем , grad_<Имя таргета>
                            # Математически: grad_X += nn_tanh_backward(X, grad_Y)
                            bwd_line = bwd_line.replace(")", f", grad_{target_tensor_name})")

                            # Перестраиваем выражение: целевым становится grad_ВходногоТензора, операция +=
                            compiled_bwd = f"if (tid < {inner_arg}Size) {{ grad_{inner_arg} += {bwd_line.split('=', 1)[1]}; }}"
                    else:
                        # Если это базовая операция типа унарного/бинарного плюс/минус/умножить без функций nn_
                        # Реализуем фолбэк накопления градиента ошибки
                        compiled_bwd = f"if (tid < {target_tensor_name}Size) {{ grad_{target_tensor_name}[tid] += grad_{target_tensor_name}[tid]; }}"

                compiled_backward_statements.append(compiled_bwd)
            # === РЕАЛИЗАЦИЯ УМНОГО ВЕКТОРНОГО ФИЛЬТРА ЗАМЕН ДЛЯ ТЕНЗОРОВ ===
            # Набор операторов и символов из вашего ТЗ (включая пробел)
            operators = [' ', '-', '+', '=', ',', '(', '/', '%', ')']

            # Обходим уникальные имена тензоров, обнаруженные в строке
            for t in detected_tensors_set:
                # 1. Формируем вектор пар: имя + оператор/символ
                pattern_vector = [f"{t}{op}" for op in operators]

                # 2. Формируем параллельный вектор замен: имя[tid] + оператор/символ
                replacement_vector = [f"{t}[tid]{op}" for op in operators]

                # 3. Выполняем алгоритм контекстной замены:
                # Пока комбинация имя+оператор встречается в строке, производим замену
                for pattern, replacement in zip(pattern_vector, replacement_vector):
                    while pattern in cleaned_line:
                        cleaned_line = cleaned_line.replace(pattern, replacement)

            # Оборачиваем валидную строку в Си-конструкцию с проверкой границ
            compiled_statement = f"if (tid < {target_tensor_name}Size) {{ {cleaned_line}; }}"
            compiled_statements.append(compiled_statement)

            # Собираем входные тензоры-аргументы (все задействованные тензоры минус те, что перезаписаны)
            for t in detected_tensors_set:
                if t in caller_locals and caller_locals[t].grad_buffer is not None:
                    if t not in self._captured_outputs:
                        self._captured_inputs.add(t)

            # --- СБОРКА ФИНАЛЬНОГО ТЕКСТА КЕРНЕЛА ---
            # СТРОКА 509: НАЧАЛО КРИТИЧЕСКОГО ИЗМЕНЕНИЯ РАЗДЕЛЕНИЯ АРГУМЕНТОВ
            kernel_hash = abs(hash(program_text))
            kernel_name = f"text_forward_{kernel_hash}"
```

```
507
508         # --- СБОРКА ФИНАЛЬНОГО ТЕКСТА КЕРНЕЛА ---
509         # СТРОКА 509: НАЧАЛО КРИТИЧЕСКОГО ИЗМЕНЕНИЯ РАЗДЕЛЕНИЯ АРГУМЕНТОВ
510         kernel_hash = abs(hash(program_text))
511         kernel_name = f"text_forward_{kernel_hash}"
512
513         # 1. ЧИСТЫЙ ЧЕТКИЙ НАБОР ДЛЯ FORWARD (ТОЛЬКО ДАННЫЕ И РАЗМЕРЫ)
514         kernel_args_forward = []
515         cl_buffers_call_fwd = []
516         max_elements = 1
517
518         # 2. ОТДЕЛЬНЫЙ СТРОГИЙ НАБОР ДЛЯ BACKWARD (ДАННЫЕ + РАЗМЕРЫ + ГРАДИЕНТЫ)
519         kernel_args_backward = []
520         cl_buffers_call_bwd = []
521
522         for tensor_name in sorted(list(detected_tensors_set)):
523             tensor_obj = caller_locals[tensor_name]
524             if tensor_obj.size > max_elements:
525                 max_elements = tensor_obj.size
526
527             # Базовые аргументы для Forward
528             kernel_args_forward.append(f"__global float* {tensor_name}")
529             kernel_args_forward.append(f"const int {tensor_name}Size")
530             cl_buffers_call_fwd.append(tensor_obj.data_buffer)
531             cl_buffers_call_fwd.append(np.int32(tensor_obj.size))
532
533             # Изолированные аргументы для Backward (формируются только если обучаемся)
534             if self.is_training:
535                 kernel_args_backward.append(f"__global float* {tensor_name}")
536                 kernel_args_backward.append(f"const int {tensor_name}Size")
537                 kernel_args_backward.append(f"__global float* grad_{tensor_name}")
538
539                 cl_buffers_call_bwd.append(tensor_obj.data_buffer)
540                 cl_buffers_call_bwd.append(np.int32(tensor_obj.size))
541                 cl_buffers_call_bwd.append(tensor_obj.grad_buffer)
542
543         # Компилируем и запускаем FORWARD ядро
544         args_fwd_str = ", ".join(kernel_args_forward)
545         forward_statements_str = "\n    ".join(compiled_statements)
546
547         full_forward_code = f"""
548         {BASE_CL_FUNCTIONS}
549         __kernel void {kernel_name}({args_fwd_str}) {{
550             int tid = {self.warp_size} * get_global_id(0) + get_global_id(1);
551         {forward_statements_str}
552         }}
553         """
554         total_count = int(np.ceil(max_elements / float(self.warp_size)) * self.warp_size)
555         global_size = (total_count // self.warp_size, self.warp_size)
556
557         program = JITProgramCache.get_or_compile(self.ctx, full_forward_code)
558         kernel = getattr(program, kernel_name)
559         kernel(self.queue, global_size, None, *cl_buffers_call_fwd)
560         self.queue.finish()
561
```

```
            global_size = (total_count // self.warp_size, self.warp_size)

        program = JITProgramCache.get_or_compile(self.ctx, full_forward_code)
        kernel = getattr(program, kernel_name)
        kernel(self.queue, global_size, None, *cl_buffers_call_fwd)
        self.queue.finish()

        # --- ПОЛНОСТЬЮ ИЗОЛИРОВАННАЯ СБОРКА И КЭШИРОВАНИЕ BACKWARD ---
        if self.is_training and compiled_backward_statements:
            backward_kernel_name = f"text_backward_{kernel_hash}"

            # Применяем векторный фильтр замен [tid] для строк backward
            processed_bwd_statements = []
            operators = [' ', '-', '+', '=', ',', '(', '/', '%', ')']
            for bwd_stmt in compiled_backward_statements:
                for t in detected_tensors_set:
                    for target_token in [t, f"grad_{t}"]:
                        pattern_vector = [f"{target_token}{op}" for op in operators]
                        replacement_vector = [f"{target_token}[tid]{op}" for op in operators]
                        for pattern, replacement in zip(pattern_vector, replacement_vector):
                            while pattern in bwd_stmt:
                                bwd_stmt = bwd_stmt.replace(pattern, replacement)
                processed_bwd_statements.append(bwd_stmt)

            backward_statements_str = "\n    ".join(processed_bwd_statements)
            args_bwd_str = ", ".join(kernel_args_backward)

            full_backward_code = f"""
            {BASE_CL_FUNCTIONS}
            __kernel void {backward_kernel_name}({args_bwd_str}) {{
                int tid = {self.warp_size} * get_global_id(0) + get_global_id(1);
                {backward_statements_str}
            }}
            """
            # Делаем снапшот изолированных backward буферов
            captured_buffers = list(cl_buffers_call_bwd)
            captured_global_size = global_size
            captured_code = full_backward_code

            # Создаем чистый ленивый callback для AutogradContext
            def runner_step_backward_callback():
                        bwd_program = JITProgramCache.get_or_compile(self.ctx, captured_code)
                        bwd_kernel = getattr(bwd_program, backward_kernel_name)
                        bwd_kernel(self.queue, captured_global_size, None, *captured_buffers)
                        self.queue.finish()

                    # Извлекаем сохраненную ссылку на autograd_ctx
                    active_ctx = self.autograd_ctx
                    if active_ctx is None:
                        # Фолбэк автодетекции, если почему-то забыли передать в init
                        for out_name in self._captured_outputs:
                            t_obj = caller_locals[out_name]
                            if hasattr(t_obj, 'autograd_ctx') and t_obj.autograd_ctx is not None:
                                active_ctx = t_obj.autograd_ctx
                                break

                if active_ctx is not None:
```

```
            # Создаем чистый ленивый callback для AutogradContext
            def runner_step_backward_callback():
                        bwd_program = JITProgramCache.get_or_compile(self.ctx, captured_code)
                        bwd_kernel = getattr(bwd_program, backward_kernel_name)
                        bwd_kernel(self.queue, captured_global_size, None, *captured_buffers)
                        self.queue.finish()

                    # Извлекаем сохраненную ссылку на autograd_ctx
                    active_ctx = self.autograd_ctx
                    if active_ctx is None:
                        # Фолбэк автодетекции, если почему-то забыли передать в init
                        for out_name in self._captured_outputs:
                            t_obj = caller_locals[out_name]
                            if hasattr(t_obj, 'autograd_ctx') and t_obj.autograd_ctx is not None:
                                active_ctx = t_obj.autograd_ctx
                                break

            if active_ctx is not None:
                active_ctx.register_backward(runner_step_backward_callback)
            else:
                self.backward_stack.append((captured_code, backward_kernel_name, captured_buffers, captured_global_size))
    # СТРОКА 575: КОНЕЦ ИЗМЕНЯЕМОГО БЛОКА КЛАССА LOCALRUNNER
```